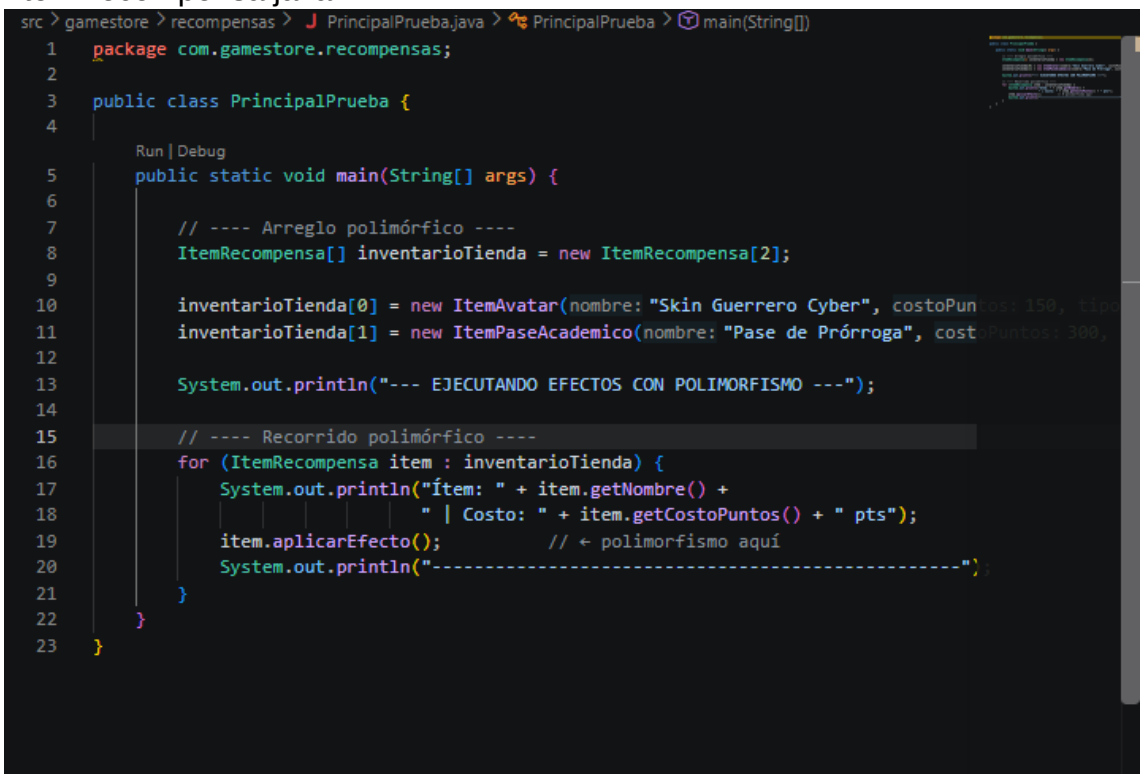
## PASO 1: CREACIÓN DEL MOLDE GENERAL (CLASE ABSTRACTA)

Primero, creé el archivo ItemRecompensa.java. Lo definí como una clase abstract porque es el molde base para todas las recompensas de la tienda. Aquí declaré los atributos privados nombre y costoPuntos, y armé su constructor. Lo más importante fue declarar el método aplicarEfecto() como abstracto (sin llaves, solo con punto y coma), lo que obliga a que cualquier recompensa que se cree a futuro tenga que definir cómo se aplica ese efecto.

ItemRecompensa.java

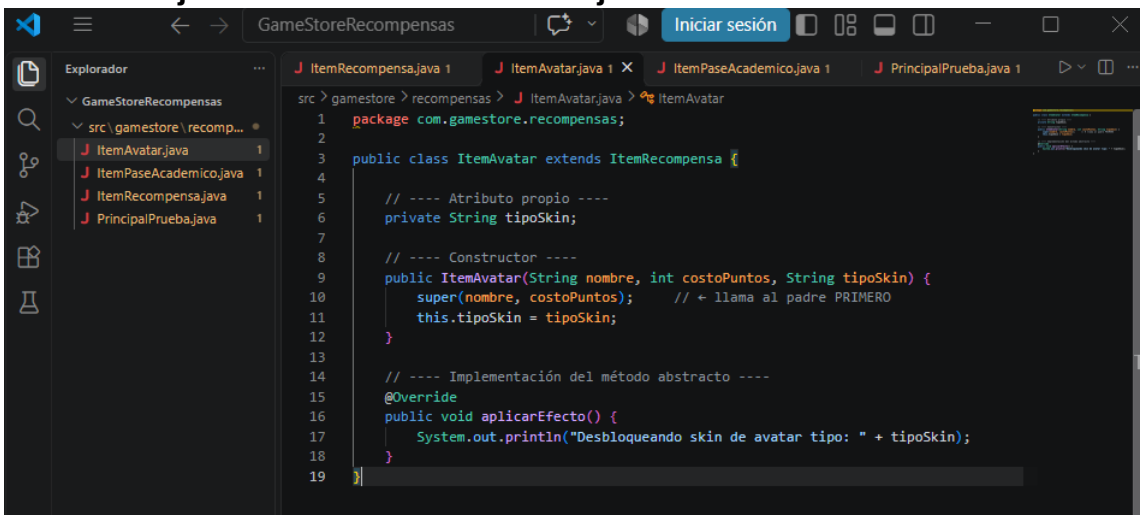


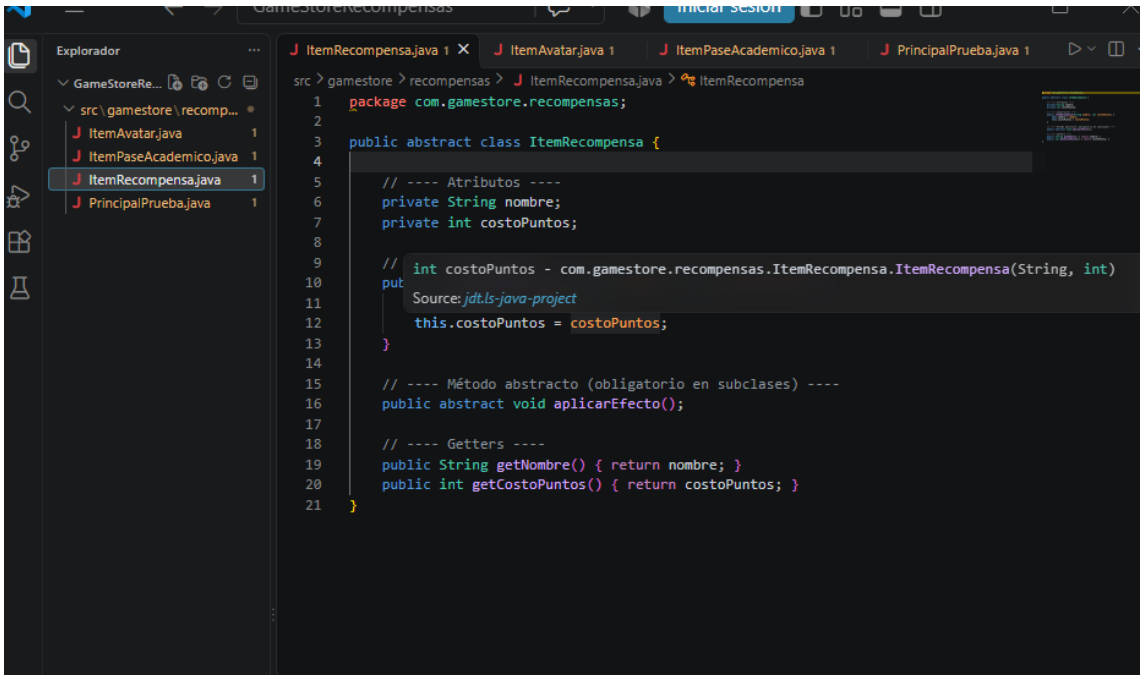

## PASO 2: CREACIÓN DE LAS RECOMPENSAS ESPECÍFICAS (SUBCLASES)

Luego, creé los archivos para las dos recompensas reales: ItemAvatar.java e ItemPaseAcademico.java. En ambos usé la palabra clave extends para heredar lo que ya había hecho en ItemRecompensa.

En el constructor de cada uno tuve que usar super(nombre, costoPuntos) en la primera línea, porque la clase padre exigía esos datos. Además, agregué sus propios atributos (el tipo de skin y los días de duración). Finalmente, utilicé @Override para implementar el método aplicarEfecto() en cada clase, poniendo un mensaje distinto para cada tipo de recompensa.

**ItemAvatar.java e ItemPaseAcademico.java**



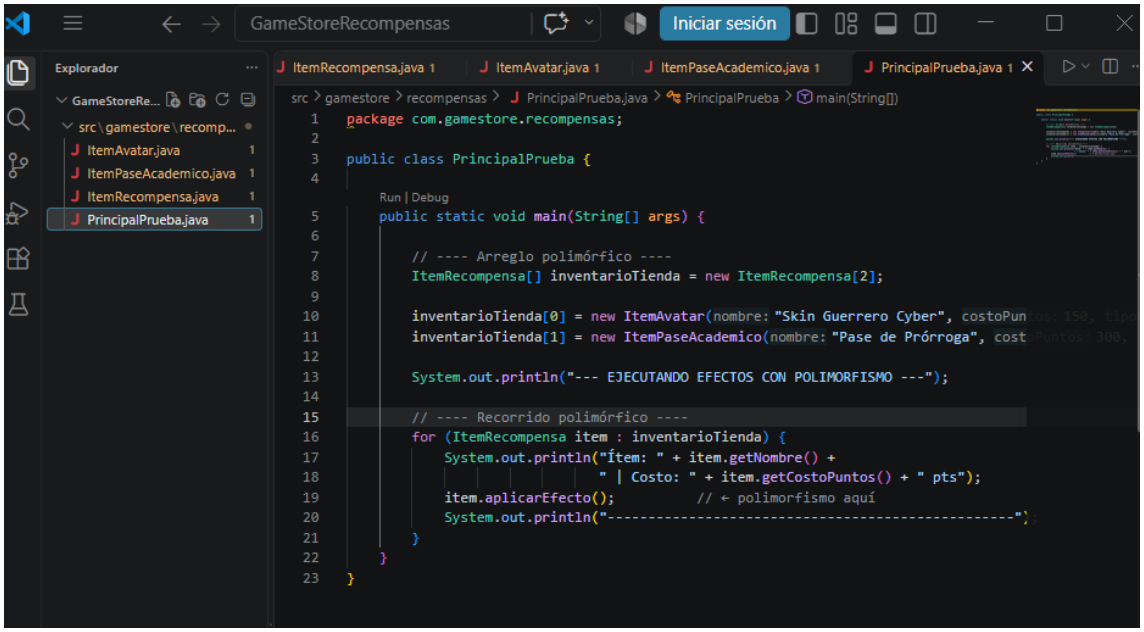




## PASO 3: IMPLEMENTACIÓN DEL POLIMORFISMO (CLASE PRINCIPAL)

Después, creé el archivo PrincipalPrueba.java con el método main para ejecutar todo. Aquí creé un arreglo de tipo ItemRecompensa[] (usando el tipo de la clase abstracta) y guardé un objeto Avatar y un objeto Pase dentro del mismo arreglo.

Para probar el polimorfismo, hice un bucle for-each para recorrer ese arreglo. Dentro del bucle, llamé al método item.aplicarEfecto(). Aquí pude comprobar la magia de la POO: aunque el bucle llama al mismo método para ambos, el Avatar ejecuta su propia versión y el Pase ejecuta la suya, imprimiendo mensajes distintos.

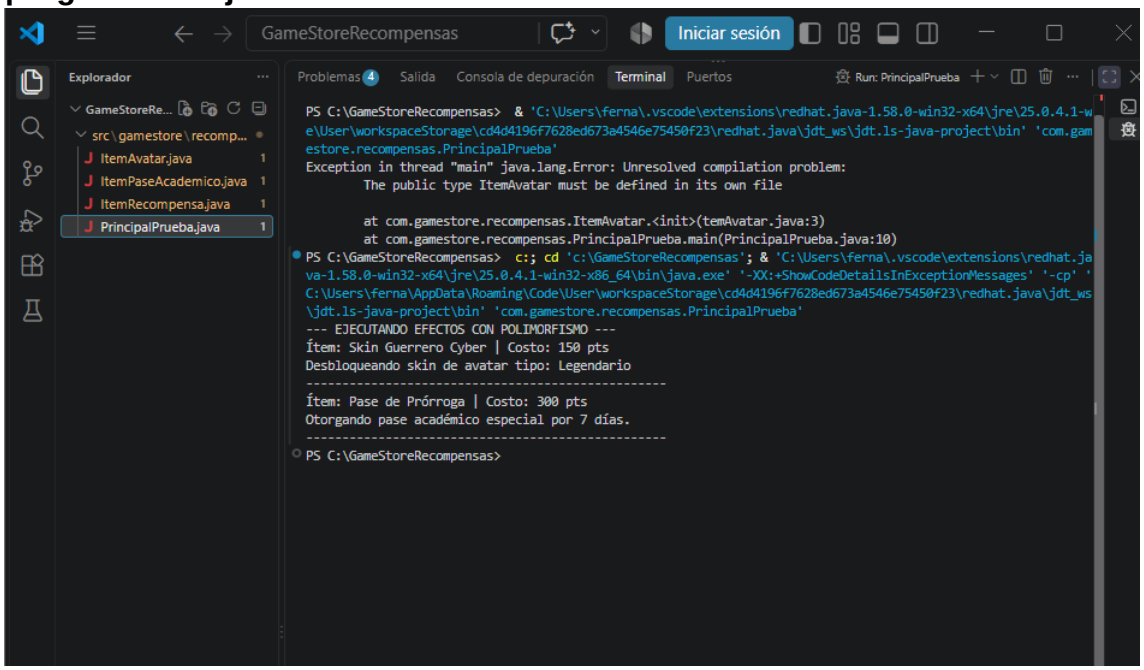
**PrincipalPrueba.java**




## PASO 4: EJECUCIÓN Y PRUEBA EN CONSOLA

Finalmente, guardé todos los archivos y ejecuté el programa desde el botón "Run" de Visual Studio Code. En la terminal de abajo se puede observar que el código compiló sin errores y se imprimieron en pantalla los datos de los ítems y sus efectos correspondientes, demostrando que el arreglo polimórfico funciona correctamente.

**programa en ejecución**



Para terminar, redacté la explicación teórica que pide la guía. Expliqué que no se puede hacer un new ItemRecompensa() porque al ser una clase abstracta, le faltan partes por definir (el cuerpo de aplicarEfecto) y Java no sabría qué hacer. Además, mencioné que usar @Override es una gran ventaja porque le avisa al compilador que estamos sobrescribiendo un método; si nos equivocamos escribiendo el nombre, el programa nos avisa con un error en lugar de crear un método nuevo por accidente.